[Insertar logo institucional]

Servicio Nacional de Aprendizaje — SENA
Centro de Electricidad, Electrónica y Telecomunicaciones — CEET
Tecnólogo en Análisis y Desarrollo de Software (ADSO) · Trimestre 6

CAPÍTULO II · GC-01

Plan de Gestión de Configuración y Control de Versiones

GitFlow · Conventional Commits · Versionamiento semántico (SemVer 2.0.0)

FASE: PLANEACIÓN

Proyecto formativo	EyeSchool — Sistema de control de asistencia escolar por QR y seguimiento académico
Ficha	3278641
Equipo / Autores	Andres Felipe Acero Lopez, Mariana Valentina Cortes Castillo, Samuel Stiven Sierra Rodriguez y Sergio Duvan Gomez Rodriguez
Instructor(es)	[Nombre del instructor técnico]
Criterio de evaluación	Trimestre 6 — En el proyecto se evidencia el uso de sistemas de control de versiones.
Versión del documento	0.2
Fecha	30/10/2026
Ciudad	Bogotá D.C.

Control del documento

Historial de versiones

Versión	Fecha	Autor(es)	Descripción del cambio
0.1	30/10/2026	Acero, Cortes, Sierra y Gomez	Creación del documento a partir de la plantilla
0.2	30/10/2026	Acero, Cortes, Sierra y Gomez	Plan diligenciado para los repositorios de la app móvil, la API y el panel web.
[1.0]			Versión aprobada

Revisión y aprobación

Rol	Nombre	Firma	Fecha
Elaboró (aprendiz / equipo)	Andres Felipe Acero Lopez, Mariana Valentina Cortes Castillo, Samuel Stiven Sierra Rodriguez y Sergio Duvan Gomez Rodriguez		
Revisó (líder técnico / Scrum Master)			
Aprobó (instructor / Product Owner)			

Convenciones de esta plantilla

[Texto resaltado]	Campo que el equipo debe diligenciar. Reemplace el texto y elimine el resaltado.
Instrucción	Orientación para diligenciar la sección. Elimine estos recuadros en la versión final.
DATOS DE EJEMPLO	Gráficas ilustrativas. Reemplácelas por las generadas con los datos reales del proyecto.

Ubicación en el paquete documental

Documento que lo precede	MC-02 Matriz de verificación de calidad
Documento que lo sucede	GC-02 Informe de evidencias del repositorio
Norma / referente principal	IEEE 828-2012 · ISO/IEC/IEEE 12207 §6.3.5 · SemVer 2.0.0 · Conventional Commits 1.0.0

Contenido

1. Introducción 4
1.1 Propósito 4
1.2 Glosario 4
2. Herramientas y repositorios 4
3. Identificación de elementos de configuración 5
4. Estrategia de ramificación: GitFlow 6
4.1 Reglas de protección de ramas 6
4.2 Flujo de un cambio 7
5. Convención de commits: Conventional Commits 1.0.0 7
6. Versionamiento semántico (SemVer 2.0.0) 8
7. Control de cambios 9
7.1 Plantilla de Pull Request 9
8. Líneas base y liberaciones 9
9. Auditoría de configuración 10
10. Archivos mínimos del repositorio 10
11. Firmas 10

Si la tabla de contenido aparece vacía: clic derecho sobre ella → Actualizar campo → Actualizar toda la tabla.

1. Introducción

1.1 Propósito

Establecer cómo el equipo identifica, controla, versiona, audita y libera los elementos de configuración del sistema EyeSchool, garantizando trazabilidad entre requisitos, código, pruebas y versiones liberadas.

1.2 Glosario

Término	Definición
Elemento de configuración (EC)	Artefacto bajo control de versiones cuyo cambio debe ser gestionado (código, scripts BD, documentos, configuración).
Línea base (baseline)	Estado aprobado de un conjunto de EC que sirve de referencia; en Git se materializa con un tag.
Rama (branch)	Línea de desarrollo independiente dentro del repositorio.
Pull Request (PR)	Solicitud de integración de cambios sujeta a revisión y validación automática.
Tag	Etiqueta inmutable que marca una versión liberada (p. ej. v1.2.0).

2. Herramientas y repositorios

Tabla 1. *Repositorios y herramientas*

Elemento	Herramienta / URL	Visibilidad	Administrador
Repositorio API (backend)	https://github.com/AndresFelipeAceroLopez/backend-Eyes-School	Público	AndresFelipeAceroLopez
Repositorio web	https://github.com/marianacortes0/proyecto-eyesschool	Público	marianacortes0
Repositorio móvil	https://github.com/AndresFelipeAceroLopez/eyes_school_app_movil	Público	AndresFelipeAceroLopez
Scripts de base de datos / migraciones	Repositorio de la API: alembic/versions/, schema.sql y seed_data.sql	Público	AndresFelipeAceroLopez
Documentación	App: planeacion/calidad/ y planeacion/control-versiones/. API: docs/. Web: planeacion/	Público	AndresFelipeAceroLopez; marianacortes0
Tablero de trabajo (issues)	[Confirme el tablero del equipo: GitHub Projects, Jira u otro. Hoy los tres repositorios tienen 0 issues]	[Público / Privado]	[Administrador]
CI/CD	GitHub Actions: ci.yml en cada repositorio, release_apk.yml (app) y main_*.yml (Azure, API y web). La web también genera vistas previas en Vercel.	Público	AndresFelipeAceroLopez; marianacortes0

El proyecto usa multirepositorio: un repositorio para la API, uno para la web y uno para la app. Cada componente tiene su propio lenguaje, sus herramientas y su forma de publicar (Azure para la API y la web; APK con actualización OTA para la app), así que un cambio en uno no dispara el CI de los otros. El contrato común es la API REST /api/v1. El costo es que las versiones y la evidencia se gestionan por separado en cada repositorio.

3. Identificación de elementos de configuración

Tabla 2. *Elementos de configuración*

Tipo de EC	Ejemplos	Ubicación	Convención de nombres
Código fuente	Pantallas, servicios, repositorios, routers, componentes	App: lib/. API: app/ y main.py. Web: src/	Dart y Python: archivos en snake_case. TypeScript: componentes en PascalCase y hooks useX
Pruebas	Unitarias, de widgets, de repositorios y de autorización	App: test/. API y web: tests/	`*_test.dart · test_*.py · *.test.ts`
Base de datos	Migraciones Alembic, esquema y datos semilla	API: alembic/versions/, schema.sql, seed_data.sql	<hash>_descripcion.py (generado por Alembic)
Configuración	.env.example, pubspec.yaml, requirements.txt, pyproject.toml, package.json, next.config.ts	Raíz de cada repositorio	`Nunca versionar secretos reales`
Pipeline	Workflows CI/CD	/.github/workflows	ci.yml, release_apk.yml, main_*.yml
Documentación	Este paquete, README, CHANGELOG	App: planeacion/. API: docs/. Web: planeacion/	`CODIGO_Nombre.docx (la versión va en el historial del documento)`

4. Estrategia de ramificación: GitFlow

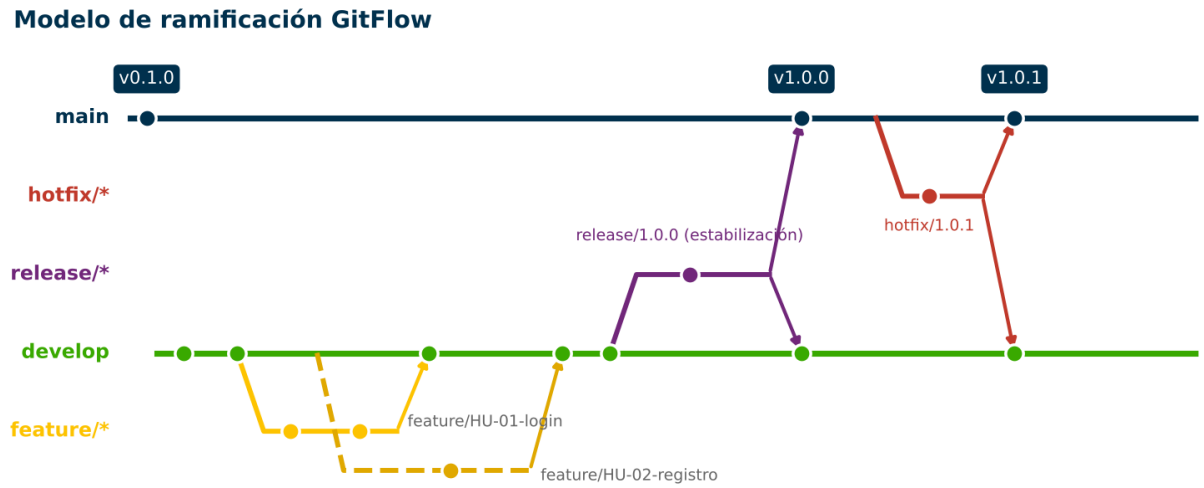


Figura 1. *Modelo de ramificación GitFlow aplicado al proyecto* Fuente: Adaptado de V. Driessen, A successful Git branching model (2010)

Tabla 3. *Ramas de GitFlow*

Rama	Origen	Se integra en	Propósito	Vida
`main`	—	—	Código en producción. Cada commit es una versión liberada con tag.	Permanente
`develop` (`developer` en la API)	main	release/*	Integración del trabajo del sprint.	Permanente
`feature/HU-xx-desc`	develop	develop	Desarrollo de una historia de usuario.	Temporal
`fix/desc` · `chore/desc`	develop	develop	Corrección o mantenimiento que no es urgente en producción.	Temporal
`release/X.Y.0`	develop	main y develop	Estabilización, pruebas de aceptación y ajuste de versión.	Temporal
`hotfix/X.Y.Z`	main	main y develop	Corrección urgente en producción.	Temporal

Se conserva GitFlow porque el proyecto libera versiones por hitos, no por despliegue continuo. La rama de integración de la API se llama developer y no se renombra, para no romper sus reglas y flujos existentes. Las ramas release/* y hotfix/* aún no se han usado; se adoptan desde la próxima versión.

4.1 Reglas de protección de ramas

- ☐ `main` y `develop` no admiten push directo; todo cambio entra por Pull Request.
- ☐ Se requiere al menos 1 aprobación de revisión de código.

☐ El pipeline CI (análisis estático, pruebas, build y auditoría de dependencias) debe estar en verde para fusionar.

☐ Se exige historial lineal (squash merge) en `develop`.

☐ Las ramas temporales se eliminan tras la fusión.

Estado al 30/10/2026: estas reglas son la meta. En la app están activas, pero develop exige 0 aprobaciones; en la API y la web existen, exigen 2 aprobaciones y están desactivadas. El GC-02 detalla el cumplimiento.

4.2 Flujo de un cambio

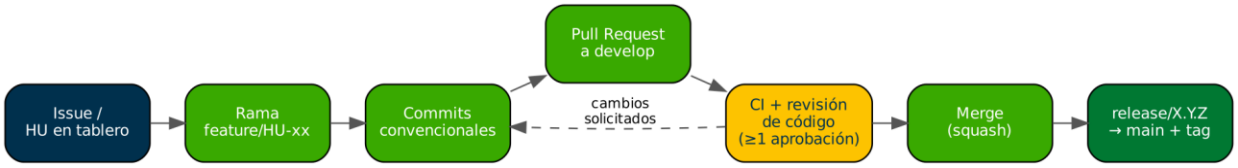


Figura 2. *Flujo de un cambio desde la historia de usuario hasta la versión*

5. Convención de commits: Conventional Commits 1.0.0



Figura 3. *Anatomía de un mensaje de commit convencional* Fuente: conventionalcommits.org

Tabla 4. *Tipos de commit*

Tipo	Uso	Efecto en SemVer
feat	Nueva funcionalidad para el usuario	MENOR
fix	Corrección de un defecto	PARCHE
docs	Solo documentación	—
style	Formato sin cambio de lógica	—
refactor	Reestructuración sin cambio de comportamiento	—
perf	Mejora de rendimiento	PARCHE
test	Agrega o corrige pruebas	—
build / ci	Sistema de construcción o pipeline	—

Tipo	Uso	Efecto en SemVer
chore	Tareas de mantenimiento	—
BREAKING CHANGE o !	Cambio incompatible (en cualquier tipo)	MAYOR

Ejemplos válidos:

```
git log
feat(ota): add github actions and auto-update mechanism
fix(auth): crear el perfil de docente y administrador al registrarse
test(auth): agregar pruebas de expiración de token
docs(readme): documentar variables de entorno
feat(api)!: paginar la respuesta de /estudiantes

Refs: HU-07
```

Hoy el formato se valida con revisión manual en cada PR, con una meta de al menos 90 % de commits conformes. La validación automática con commitlint en el CI y el CHANGELOG generado con release-please son acciones de mejora registradas en el GC-02.

6. Versionamiento semántico (SemVer 2.0.0)



Figura 4. *Estructura del número de versión semántica* Fuente: semver.org

Tabla 5. *Ejemplos de incremento de versión*

Situación	Versión anterior	Nueva versión
Primera versión liberada a producción	0.9.0	1.0.0
Se agrega un módulo de reportes compatible	1.0.0	1.1.0
Se corrige un error en el cálculo de totales	1.1.0	1.1.1
Se elimina un campo obligatorio de la API	1.1.1	2.0.0

Situación	Versión anterior	Nueva versión
Candidata a liberación para UAT	1.2.0	1.2.0-rc.1

Las versiones de API, web y móvil se versionan de forma independiente. La aplicación móvil además maneja `versionCode` incremental exigido por las tiendas.

Versiones actuales: app 1.0.1, API 0.1.0 y web 0.1.0. En la app, release_apk.yml toma la versión del tag y usa el número de ejecución del workflow como versionCode.

7. Control de cambios

Toda solicitud de cambio se registra como issue con: descripción, justificación, impacto, prioridad y HU relacionada. Los cambios a una línea base liberada requieren aprobación del Product Owner.

7.1 Plantilla de Pull Request

```
.github/pull_request_template.md
## Descripción
Qué cambia y por qué. Refs: HU-xx / #issue

## Tipo de cambio
- [ ] feat   - [ ] fix   - [ ] refactor   - [ ] docs   - [ ] BREAKING CHANGE

## ¿Cómo se probó?
- [ ] Pruebas unitarias   - [ ] Pruebas de integración   - [ ] Prueba manual

## Checklist
- [ ] El pipeline está en verde
- [ ] Análisis estático y auditoría de dependencias sin errores
- [ ] Se actualizó la documentación
```

8. Líneas base y liberaciones

Línea base	Contenido	Tag	Fecha prevista	Aprobó
LB-Requisitos	Backlog y HU aprobadas	req-v1	[dd/mm/aaaa]	Sergio Duvan Gomez Rodriguez
LB-Diseño	Arquitectura y modelo de datos	design-v1	[dd/mm/aaaa]	Sergio Duvan Gomez Rodriguez
LB-Release 1	Código + BD + docs de la versión	App: v1.0.1 (ya publicada el 17/09/2026). API y web: v1.0.0 (pendiente)	[dd/mm/aaaa]	Sergio Duvan Gomez Rodriguez

9. Auditoría de configuración

- ☐ Todos los tags corresponden a commits en `main` con pipeline exitoso.
- ☐ El CHANGELOG refleja los commits feat/fix de cada versión.
- ☐ No existen secretos versionados (verificar con gitleaks o secret scanning).
- ☐ Cada PR fusionado referencia una HU o issue.
- ☐ El archivo `.gitignore` excluye binarios, dependencias y archivos de entorno.

10. Archivos mínimos del repositorio

```
estructura
proyecto/
├── .github/
│   ├── workflows/ci.yml
│   └── pull_request_template.md
├── lib/ | app/ | src/     # código: app | API | web
├── test/ | tests/         # pruebas: app | API y web
├── alembic/               # migraciones (solo API)
├── docs/ | planeacion/    # documentación
├── .gitignore
├── .env.example           # API y web
├── CHANGELOG.md
├── CONTRIBUTING.md        # convenciones de ramas y commits
├── LICENSE
└── README.md
```

11. Firmas

[Nombre completo]	[Nombre completo]
Elaboró	**Aprobó**
Fecha: [dd/mm/aaaa]	Fecha: [dd/mm/aaaa]